

ابن موقعك الأول مع Claude Code

دليل عملي لبناء موقع حقيقي من الفكرة إلى
النشر باستخدام Claude Code

مشروع ويب خطوة بخطوة

كل
موقع
يبدأ
بفكرة



برومبتات عربية

جميع الحقوق محفوظة لقناة برومبتات عربية.

هذا المنتج مخصص للاستخدام الشخصي للمشتري. لا يجوز إعادة نشره أو بيعه أو توزيعه أو اقتباس أجزاء منه لأغراض تجارية دون إذن كتابي من المالك.

القناة الرسمية: <https://t.me/PromptsArabic>

اسم المنتج	ابن موقعك الأول مع Claude Code
الإصدار	الإصدار الأول
اللغة	العربية
المقاس	A4 عمودي — مهيأ للقراءة على الهاتف
تمت مراجعة معلومات وميزات 6 سبتمبر 2026 هذا الدليل حتى تاريخ	
مصادر التحقق	code.claude.com/docs و nodejs.org و react.dev و nextjs.org

ملاحظة مهمة عن تاريخ المحتوى

منصات الذكاء الاصطناعي تتغير بسرعة. كل ميزة مذكورة هنا تم التحقق منها من المصدر الرسمي بتاريخ 6 سبتمبر 2026. إذا مر وقت طويل على هذا التاريخ، راجع الصفحة الرسمية للميزة قبل الاعتماد عليها في عمل حساس.

عن هذا الدليل

معظم أدلة بناء المواقع تعطيك أوامر تنسخها فتحصل على شيء يعمل ولا تعرف لماذا. هذا الدليل يبني معك موقعا عربيا حقيقيا على عشر مراحل، ولكل مرحلة: ما نبنيه، وما نرسله إلى Claude Code حرفيا، وما الملفات المتوقعة، وكيف تختبر بنفسك، وما الأخطاء المحتملة، ومتى تنتقل للمرحلة التالية. المشروع الناتج ليس تمرينا: دليل خدمات محلي بواجهة عربية RTL وبحث ونموذج تواصل، جاهز للنشر.

لمن هذا الدليل

- من يعرف أساسيات البرمجة ولم يبن موقعا كاملا من قبل
- طلاب يحتاجون مشروع ويب حقيقيا لا صفحة واحدة
- من بنى مواقع بالنسخ واللصق ويريد فهم القرارات
- من يريد موقعا عربيا صحيح الاتجاه لا موقعا إنجليزيا مترجما

ماذا ستتعلم

- كيف تحول فكرة غامضة إلى نطاق محدد وقصص مستخدم قابلة للبناء
- كيف تقارن أطر العمل وتختار واحدا بأسباب لا بشهرة
- بناء موقع بالمراحل مع أوامر فعلية ترسلها إلى Claude Code
- طبقة بيانات نظيفة تسمح لك بتغيير مصدر البيانات لاحقا
- نماذج آمنة بتحقق على الخادم لا على المتصفح فقط
- تصميم متجاوب عربي RTL وإمكانية وصول حقيقية
- أساسيات الأداء و SEO والنشر والصيانة

قبل أن تبدأ

المتطلبات

- Node.js — الإصدار المدعوم طويل الأمد وقت مراجعة هذا الدليل هو 24
- Claude Code مثبت ويعمل
- محرر نصوص، ويفضل VS Code
- معرفة أساسية بـ HTML و CSS و JavaScript
- حساب Git ومستودع فارغ

كيف تستخدم هذا الدليل

- لا تقفز إلى مرحلة البناء. المرحلتان الأولى والثانية هما ما يحدد جودة المشروع كله
- أرسل برومبت كل مرحلة كما هو بعد تعبئة المتغيرات، ولا تختصره
- بعد كل مرحلة: راجع الفروق ثم اختر ثم commit، بهذا الترتيب
- إذا انحرف Claude عن المطلوب، ارجع إلى برومبت المرحلة ولا تصحح بطلبات متفرقة
- احتفظ بملف القرارات وحديثه بعد كل مرحلة

طريقة القراءة الموصى بها

لا تقرأ الدليل كله دفعة واحدة. اقرأ فصلا واحدا، ثم نفذ تمرينه على مهمة حقيقية من عملك، ثم انتقل. أربعة فصول مطبقة أفضل من أربعة عشر فصلا مقروءا.

7	● من فكرة غامضة إلى نطاق محدد
7	المشروع الذي سنبنيه معا
7	ثلاثة أسئلة قبل أي شيء
8	برومبت تحديد النطاق
9	علامات نطاق سيئ
10	● المخطط الهيكلي وبنية الصفحات
10	صفحات موقعنا
10	الحالات التي ينساها المبتدئون
11	برومبت المخطط والبنية
11	شكل البيانات الذي سنستخدمه
12	● اختيار Stack بقرار لا بشهرة
12	المرشحون
13	المقارنة على معايير مشروعنا
13	القرار وسببه
14	الإصدارات التي سنعتمدها
15	● البيئة وإعداد المشروع
16	ضبط الاتجاه العربي
18	● الصفحات والتوجيه والحالات
20	● البيانات وطبقة الوصول

22	لماذا هذه الطبقة تحديدا
23	● النماذج والتحقق
25	● التصميم المتجاوب وإمكانية الوصول
27	● الأداء وأساسيات SEO
29	● الاختبارات والأمان
29	الاختبارات
29	الأمان
31	● البناء والنشر والصيانة
31	قبل النشر
31	برومبت تجهيز النشر
32	خيارات النشر
32	بعد الإطلاق
33	● ورقة المراجعة السريعة
33	المراحل العشر ومعايير الانتقال
34	دورة العمل في كل مرحلة
34	أخطاء شائعة في أول موقع
35	ما تضعه في CLAUDE.md لهذا المشروع

من فكرة غامضة إلى نطاق محدد

الفكرة الغامضة تنتج مشروعاً لا ينتهي. المشروع الذي ينتهي هو الذي عرف بدقة قبل أن يكتب سطر. هذه المرحلة لا كود فيها إطلاقاً.

المشروع الذي سنبنه معاً

سنبن دليل خدمات محلي: موقع يعرض مزودي خدمة في مدينة، يستطيع الزائر تصفحهم والبحث فيهم حسب التخصص، ويفتح صفحة لكل مزود، ويرسل طلب تواصل. اخترنا هذه الفكرة لأنها تحتوي كل ما يحتاجه أي موقع حقيقي: قائمة، وتفاصيل، وبحث، ونموذج، وبيانات، ونشر — بلا تعقيد يشنت المبتدئ.

إن كانت لديك فكرتك الخاصة

استبدلها بفكرتنا وأكمل بالمراحل نفسها. كل برومبت في هذا الدليل مكتوب بمتغيرات، فيصلح لأي موقع من هذا النوع: مطاعم، أو دورات، أو عقارات، أو أطباء، أو أي دليل يعرض عناصر ويسمح بالبحث فيها والتواصل بشأنها.

ثلاثة أسئلة قبل أي شيء

1 من المستخدم بالضبط

لا «الجميع». شخص واحد محدد بحاجة محددة.

2 ماذا يفعل اليوم بدون موقعك

إذا لم تكن هناك طريقة حالية، فالمشكلة قد لا تكون حقيقية.

3 ما الشيء الواحد الذي لو لم يفعله الموقع لكان بلا قيمة

هذا هو جوهر مشروعك، وكل ما عداه إضافة.

برومبت تحديد النطاق

أرسل هذا في جلسة Claude Code جديدة، قبل إنشاء أي ملف

سنبني موقعا. لا تكتب أي كود في هذه المرحلة.

الفكرة:

[الفكرة]

المستخدم المستهدف:

[المستخدم]

ما يفعله اليوم بدون هذا الموقع:

[البديل الحالي]

المدة المتاحة لي:

[المدة]

وقي الأسبوعي:

[الوقت الأسبوعي]

المطلوب: أنشئ ملف REQUIREMENTS.md يحتوي:

أولا. المشكلة في ثلاث جمل، بلغة المستخدم لا بلغة تقنية

ثانيا. قصص المستخدم الأساسية، خمس على الأكثر، بصيغة: كمستخدم أريد كذا حتى أستطيع كذا

ثالثا. داخل النطاق: قائمة محددة لما سيفعله الموقع

رابعا. خارج النطاق صراحة: قائمة أطول، وكل بند مع سبب استبعاده

خامسا. معايير القبول: متى نعتبر المشروع منتهيا

سادسا. الافتراضات التي لو سقطت لتغير المشروع

سابعا. ثلاثة طلبات إضافية متوقعة لاحقا، والرد المعد لكل منها

قواعد:

اجعل قسم خارج النطاق مفصلا، فهو الذي يحمي المشروع من التضخم.

إذا كانت الفكرة أوسع من مدتي المتاحة، قل ذلك واقترح تضييقها.

لا تفترض معلومة عني. إذا نقصك شيء، اسأل.

علامات نطاق سيئ

العلامة	لماذا هي خطر
قائمة داخل النطاق أطول من قائمة خارجه	لم تستبعد شيئاً فعلاً
قصة مستخدم تبدأ بـ «كمسؤول»	دخلت لوحة تحكم كاملة إلى مشروعك الأول
معياري قبول غير قابل للقياس	لن تعرف متى تتوقف
أكثر من خمس قصص أساسية	قسم المشروع إلى إصدارين

المخطط الهيكلي وبنية الصفحات

المخطط الهيكلي wireframe ليس تصميمًا. هو قرار عن ما يظهر وأين وبأي ترتيب، قبل أن تشغلك الألوان.

صفحات موقعنا

الصفحة	المسار	ما تحتويه	ما لا تحتويه
الرئيسية	/	عنوان، شريط بحث، شبكة المزودين، ترقيم صفحات	شرح طويل عن الموقع
تفاصيل مزود	/providers/[معرف]	الاسم، التخصص، الوصف، معلومات التواصل، زر الطلب	تعليقات وتقييمات
نتائج البحث	search/	الفلاتر، النتائج، حالة لا نتائج	بحث ذكي أو اقتراحات
تواصل	contact/	نموذج، رسالة نجاح، رسائل خطأ	خرائط ودردشة
صفحة 404	غير موجود	رسالة واضحة ورابط للرئيسية	أي شيء آخر

الحالات التي ينساها المبتدئون

لكل صفحة أربع حالات لا واحدة. تصميم الحالة الطبيعية فقط هو أشهر سبب لموقع يبدو مكسورا عند أول استخدام حقيقي:

- حالة التحميل: ماذا يرى المستخدم أثناء جلب البيانات؟
- الحالة الفارغة: لا نتائج بحث، أو لا مزودين بعد. ماذا يقول الموقع؟
- حالة الخطأ: فشل الجلب. رسالة مفهومة، لا شاشة بيضاء.
- الحالة الطبيعية: البيانات موجودة وتعرض.

برومبت المخطط والبنية

لا كود بعد – مخطط فقط

بناء على REQUIREMENTS.md، صمم بنية الموقع. لا تكتب كودا بعد.

المطلوب في ملف ARCHITECTURE.md:

أولا. خريطة الصفحات: المسار، والغرض، والبيانات التي تحتاجها كل صفحة
ثانيا. لكل صفحة: الحالات الأربع – تحميل، فارغة، خطأ، طبيعية – وما يعرض في كل حالة
ثالثا. المكونات المشتركة بين الصفحات، وأين تستخدم
رابعا. شكل البيانات: ما الحقول التي يحملها العنصر الواحد، ونوع كل حقل، وأيها إلزامي
خامسا. مسار المستخدم من الدخول إلى إرسال الطلب، خطوة بخطوة
سادسا. القرارات المعمارية التي يجب تثبيتها في CLAUDE.md

قواعد:

الموقع عربي واتجاهه من اليمين إلى اليسار. خذ ذلك في الحسبان في تخطيط كل صفحة.
لا تقترح ميزة خارج النطاق المتفق عليه.
إذا كان حقل بيانات غامضا، اسألني عنه بدل افتراضه.

شكل البيانات الذي سنستخدمه

types/provider.ts

```
export type Provider = {
  id: string;
  name: string;
  category: string;
  city: string;
  description: string;
  phone: string;
  verified: boolean;
};
```

ابدأ من شكل البيانات

تحديد شكل العنصر الواحد قبل بناء الصفحات يوفر عليك إعادة كتابة نصف المشروع لاحقا. كل صفحة بعد ذلك تصبح عرضا مختلفا للبنية نفسها.

اختيار Stack بقرار لا بشهرة

لا يوجد إطار عمل أفضل. يوجد إطار أنسب لحالتك. هذه المقارنة على معايير مشروعنا نحن: مبتدئ، ومشروع حقيقي، ومدة محدودة، وعمل مع أداة وكيلة.

المرشحون

الخيار	طبيعته	الحالة وقت المراجعة
Next.js	إطار React كامل: توجيه وعرض على الخادم وواجهات خلفية	الإصدار المستقر 16.3، ودعم مدمج مستقر لمصرف React
React مع Vite	مكتبة واجهة مع أداة بناء، بلا توجيه ولا خادم	React المستقر 19.0.4
Astro	إطار موجه للمحتوى، يرسل أقل قدر من JavaScript	إصدارات 6.3 و 6.4 مستقرة في 2026
SvelteKit	إطار كامل بلغة Svelte	المستقر هو الجيل الثاني؛ الجيل الثالث كان مرشح إصدار وقت المراجعة

قاعدة الإصدارات في هذا الدليل

لن نبني على إصدار مرشح release candidate ولا على بيتا. مرشح الإصدار قد تتغير واجهته قبل الاستقرار، فتجد نفسك تعيد كتابة كود يعمل. هذا هو سبب استبعاد الجيل الثالث من SvelteKit من مسارنا رغم جودته.

المقارنة على معايير مشروعنا

المعيار	Next.js	Vite مع React	Astro
سهولة البدء لمبتدئ	متوسطة: مفاهيم أكثر	عالية: أقل مفاهيم	عالية
توجيه الصفحات جاهز	نعم	لا، تضيفه بنفسك	نعم
واجهة خلفية في المشروع نفسه	نعم	لا	جزئيا
مناسب لموقع محتوى	نعم	متوسط	ممتاز
مناسب لتطبيق تفاعلي	ممتاز	ممتاز	متوسط
حجم المجتمع والأمثلة	كبير جدا	كبير جدا	متوسط
سهولة النشر	عالية	عالية	عالية

القرار وسببه

نختار **Next.js** لهذا الدليل، لثلاثة أسباب تخص مشروعنا تحديداً:

1. موقعنا يحتاج صفحات وبحثا ونموذجا يكتب بيانات. Next.js يعطي التوجيه والواجهة الخلفية في مشروع واحد، فلا نربط مشروعين ونحن في البداية.
2. التوجيه المبني على الملفات يجعل بنية المشروع مقروءة، وهذا يفيدك أنت ويفيد الأداة الوكيل التي تقرأ مشروعك.
3. حجم المجتمع يعني أن أي رسالة خطأ ستجد لها إجابة، وهذا يهم المبتدئ أكثر من أي ميزة تقنية.

ومتي لا يكون الاختيار الصحيح؟ إذا كان موقعك محتوى ثابتا بلا تفاعل ف Astro أخف وأنسب. وإذا كنت تبني لوحة تفاعلية بلا صفحات عامة ف React مع Vite أبسط.

الإصدارات التي سنعتمدها

الأداة	الإصدار	الحالة
Node.js	24 (Krypton)	مدعوم طويل الأمد نشط وقت المراجعة
Next.js	16.3	مستقر
React	19.0.4	مستقر
TypeScript	الإصدار الذي يثبتته منشئ المشروع	مستقر

قبل أن تبدأ التنفيذ

الإصدارات أعلاه تم التحقق منها من المصادر الرسمية بتاريخ 6 سبتمبر 2026. أطر الويب تتحرك بسرعة: افتح صفحة الإصدارات الرسمية وتأكد من أحدث إصدار مستقر قبل أن تبدأ، ولا تعتمد إصدارا موسوما beta أو rc أو .canary

البيئة وإعداد المشروع

المرحلة التي يتخطاها الناس فتكلفتهم ساعات لاحقا.

ما الذي نبنيه في هذه المرحلة

بيئة تعمل، ومشروع Next.js فارغ، ومستودع Git، وملف CLAUDE.md يعرف فيه Claude Code أوامر مشروعك وأعرافه.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

أنشئ مشروع Next.js جديدا في المجلد الحالي.

المتطلبات:

استخدم أحدث إصدار مستقر من Next.js. تحقق من الإصدار قبل التثبيت ولا تستخدم beta أو rc. TypeScript مفعّل.

وجه التطبيق App Router.

بنية مجلدات واضحة: app للصفحات، components للمكونات، lib للمنطق، types للأنواع.

بعد الإنشاء:

أولا. اضبط اللغة العربية والاتجاه من اليمين إلى اليسار على مستوى المستند

ثانيا. أنشئ ملف .gitignore مناسباً، ويجب أن يستبعد env و node_modules ومجلد البناء

ثالثاً. أنشئ env.example فارغ القيم

رابعا. أنشئ CLAUDE.md يحتوي أوامر التشغيل والاختبار والفحص، وأعراف التسمية، وقرار أن الموقع عربي RTL

اعرض لي الخطة والملفات التي ستنشئها قبل أن تنفذ، وانتظر موافقتي.

الملفات المتوقعة إنشاؤها أو تعديلها

• package.json و tsconfig.json و next.config

• app/layout — وفيه ضبط اللغة والاتجاه

• app/page — الصفحة الرئيسية المؤقتة

• .gitignore و .env.example

• CLAUDE.md

كيف تختبر بنفسك

1. شغل خادم التطوير وافتح الموقع في المتصفح.
2. افحص عنصر html في أدوات المطور: هل `lang` عربية و `dir` من اليمين لليسار؟
3. شغل `git status` وتأكد أن `node_modules` و `.env` غير مدرجين.
4. شغل `/context` داخل Claude Code وتأكد أن `CLAUDE.md` ظهر تحت Memory files.

أخطاء محتملة

- نسيان ضبط الاتجاه في الجذر، فتضطر لإصلاحه في كل مكون لاحقاً.
- إنشاء `.gitignore` بعد أول `commit`، فيصل `node_modules` إلى المستودع.
- ترك `CLAUDE.md` فارغاً، فتعيد شرح المشروع في كل جلسة.

معيّار الانتقال

الموقع يفتح في المتصفح، والاتجاه من اليمين لليسار، والمستودع نظيف، و `CLAUDE.md` محمل.

ضبط الاتجاه العربي

app/layout.tsx — الجزء الحاسم

```
export default function RootLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  return (
    <html lang="ar" dir="rtl">
      <body>{children}</body>
    </html>
  );
}
```

الاتجاه ليس محاذاة

ضبط `dir="rtl"` على المستند هو الاتجاه الحقيقي: تنقلب معه الهوامش المنطقية وترتيب العناصر واتجاه التمرير. أما `text-align: right` وحده فهو محاذاة شكلية تترك بقية التخطيط معكوسا. استخدم الخصائص المنطقية في CSS مثل `margin-inline-start` بدل `margin-left` حتى يعمل تخطيطك في الاتجاهين.

الصفحات والتوجيه والحالات

الآن نبنى الهيكل الظاهر: صفحات حقيقية بمسارات حقيقية، وكل صفحة بحالاتها الأربع.

ما الذي نبنيه في هذه المرحلة

الصفحات الخمس من المرحلة الأولى، بمحتوى مؤقت، مع تخطيط مشترك وتنقل يعمل، وكل صفحة تعرض حالاتها الأربع.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

ابن صفحات الموقع بناء على ARCHITECTURE.md.

المطلوب:

الصفحة الرئيسية على المسار الجذر

صفحة تفاصيل مزود بمسار ديناميكي حسب المعرف

صفحة بحث

صفحة تواصل

صفحة غير موجود

لكل صفحة:

اعرض الحالات الأربع: تحميل، وفارغة، وخطأ، وطبيعية

استخدم بيانات وهمية ثابتة الآن، وسنبدل مصدر البيانات في المرحلة التالية

مشترك:

تخطيط واحد فيه رأس بتنقل وتذييل

عناوين HTML بترتيب صحيح: عنوان رئيسي واحد لكل صفحة

كل نص بالعربية، والاتجاه من اليمين إلى اليسار

قواعد:

لا تستخدم مكتبة واجهة خارجية في هذه المرحلة.

لا تضيف ميزة خارج النطاق.

اعرض خطة الملفات قبل التنفيذ، وانتظر موافقتي.

الملفات المتوقعة إنشاؤها أو تعديلها

• app/page و app/search/page و app/contact/page و app/not-found

- `app/providers/[id]/page`
- `components/Header` و `components/Footer` و `components/ProviderCard`
- `components/EmptyState` و `components/ErrorMessage` و `components/Loading`
- `lib/mock-data`

كيف تختبر بنفسك

1. افتح كل مسار يدويا وتأكد أنه يعمل.
2. افتح مسار مزود غير موجود وتأكد من ظهور صفحة 404 لا خطأ.
3. افحص العناوين: هل يوجد عنوان رئيسي واحد فقط في كل صفحة؟
4. صغر نافذة المتصفح إلى عرض هاتف وتأكد أن لا شيء يخرج عن الشاشة أفقياً.

أخطاء محتملة

- بناء الحالة الطبيعية فقط، ثم اكتشاف أن الموقع يعرض شاشة بيضاء عند الخطأ.
- أكثر من عنوان رئيسي في الصفحة، وهو خطأ في البنية وفي إمكانية الوصول معا.
- نسخ نصوص إنجليزية مؤقتة تبقى في المشروع حتى النشر.

معيار الانتقال

كل المسارات تعمل، وكل صفحة تعرض حالاتها الأربع، ولا تمرير أفقي على الهاتف.

البيانات وطبقة الوصول

أهم قرار معماري في هذا المشروع: الصفحات لا تعرف من أين تأتي البيانات. نتحدث مع طبقة واحدة، فتستطيع تغيير المصدر لاحقاً دون لمس الصفحات.

ما الذي نبنيه في هذه المرحلة

وحدة `lib/data` تعرف واجهة الوصول للبيانات، ومصدر أولي من ملف JSON مكتوب بأنواع صحيحة، وبحث وفلتر وترقيم صفحات يعمل على هذه الطبقة.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

أنشئ طبقة وصول للبيانات.

الواجهة المطلوبة في `lib/data`:

دالة ترجع قائمة المزودين مع دعم البحث والفلتر والترقيم
دالة ترجع مزوداً واحداً بمعرفه، وترجع قيمة فارغة إن لم يوجد
دالة ترجع قائمة التصنيفات المتاحة

المصدر الآن: ملف JSON فيه عشرون مزوداً وهمياً ببيانات عربية واقعية.

قواعد التصميم:

الصفحات لا تقرأ الملف مباشرة أبداً، بل تستدعي هذه الدوال فقط.
الأنواع معرفة في `types` ومستخدمه في كل مكان، بلا `any`.
البحث لا يفرق بين حالة الأحرف، ويتجاهل المسافات الزائدة.
البحث يعمل على النص العربي بشكل صحيح.
إذا لم توجد نتائج، ترجع الدالة قائمة فارغة ولا ترفع خطأً.

ثم اربط الصفحات بهذه الطبقة بدل البيانات الوهمية المباشرة.
اعرض تصميم الواجهة قبل التنفيذ وانتظر موافقتي.

الملفات المتوقع إنشاؤها أو تعديلها

• `types/provider`

• `lib/data` — الواجهة والتنفيذ

- `data/providers.json`
- تعديل صفحات الرئيسية والبحث والتفاصيل لتستخدم الطبقة

كيف تختبر بنفسك

1. ابحث عن كلمة موجودة وتأكد من النتائج.
2. ابحث عن كلمة غير موجودة وتأكد من ظهور الحالة الفارغة لا خطأ.
3. افتح مزودا بمعرف غير موجود وتأكد من صفحة 404.
4. ابحث بحروف عربية مع مسافة زائدة في البداية والنهاية.
5. ابحث في `app/` عن أي قراءة مباشرة لملف JSON – يجب ألا توجد.

أخطاء محتملة

- قراءة ملف البيانات داخل مكون صفحة، فتفقد فائدة الطبقة كلها.
- استخدام `any` في الأنواع، فتفقد فائدة TypeScript كلها.
- بحث يعمل بالإنجليزية ويفشل مع العربية بسبب تطبيع النص.

معييار الانتقال

تستطيع تغيير مصدر البيانات في ملف واحد دون تعديل أي صفحة.

```
import type { Provider } from "@types/provider";

export type ListOptions = {
  query?: string;
  category?: string;
  page?: number;
  perPage?: number;
};

export type ListResult = {
  items: Provider[];
  total: number;
  page: number;
  perPage: number;
};

export async function listProviders(options: ListOptions): Promise<ListResult> {
  // implementation reads from the current source
}

export async function getProvider(id: string): Promise<Provider | null> {
  // implementation reads from the current source
}
```

الترقية إلى قاعدة بيانات لاحقا

عندما تكبر بياناتك، تستبدل تنفيذ هاتين الدالتين بقاعدة بيانات دون أن تلمس صفحة واحدة. وإن أردت SQLite، انتبه إلى أن وحدة `node:sqlite` المدمجة في Node كانت مصنفة مرشح إصدار وقت مراجعة هذا الدليل، فتتحقق من حالتها الحالية أو استخدم مشغلا مستقرا.

النماذج والتحقق

النموذج هو المكان الوحيد في موقعك الذي يكتب فيه شخص من الخارج بيانات. ولهذا هو أول مكان يجب تأمينه.

ما الذي نبنيه في هذه المرحلة

نموذج تواصل يعمل، يتحقق على المتصفح وتحقق مستقل على الخادم، ورسائل خطأ عربية واضحة، وحماية أساسية من الإرسال المتكرر.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

ابن نموذج التواصل.

الحقول: الاسم، والبريد، ورقم الهاتف اختياري، والرسالة، ومعرف المزود إن جاء من صفحته.

التحقق:

تحقق في المتصفح لتحسين التجربة فقط
تحقق كامل ومستقل على الخادم، ولا تثق بأي تحقق جاء من المتصفح
قواعد: الاسم بين حرفين ومئة حرف، البريد بصيغة صحيحة، الرسالة بين عشرة أحرف وألفين

السلوك:

عند النجاح: رسالة نجاح واضحة وتفرغ النموذج
عند الفشل: رسالة خطأ بجانب الحقل المسؤول، بالعربية
أثناء الإرسال: تعطيل زر الإرسال ومنع الإرسال المزدوج

الحفظ: اكتب الطلب في ملف أو تخزين بسيط عبر طبقة البيانات، لا مباشرة من المكون.

قواعد أمنية:

لا تعرض تفاصيل داخلية في رسالة الخطأ للمستخدم
عقم المدخلات قبل عرضها في أي مكان
لا تضع أي سر في كود المتصفح

اعرض الخطة قبل التنفيذ.

الملفات المتوقعة إنشاؤها أو تعديلها

• `app/contact/page` و مكون النموذج

- معالجة إرسال على الخادم
- `lib/validation` – قواعد التحقق مشتركة بين الطرفين
- تعديل `lib/data` لإضافة حفظ الطلب

كيف تختبر بنفسك

1. أرسل النموذج فارغا وتأكد من ظهور رسائل خطأ عربية بجانب كل حقل.
2. أرسل بريدا غير صحيح وتأكد من الرفض.
3. أرسل نموذجا صحيحا وتأكد من حفظ الطلب فعلا.
4. عطل JavaScript في المتصفح وجرب الإرسال: هل ما زال التحقق يعمل على الخادم؟
5. اضغط زر الإرسال مرتين بسرعة وتأكد من عدم حفظ طلبين.

أخطاء محتملة

- الاكتفاء بتحقيق المتصفح – يمكن تجاوزه بالكامل في ثوان.
- رسالة خطأ عامة لا تقول للمستخدم أي حقل المشكلة.
- عرض نص المستخدم كما هو في صفحة أخرى دون تعقيم.

معييار الانتقال

النموذج يعمل، والتحقق على الخادم يرفض المدخلات السيئة حتى مع تعطيل JavaScript.

التصميم المتجاوب وإمكانية الوصول

أغلب زوار موقع عربي سيأتون من الهاتف. والتصميم المتجاوب ليس تصغير الشاشة، بل إعادة ترتيب المحتوى بحسب المساحة.

ما الذي نبنيه في هذه المرحلة

تخطيط يعمل من عرض 320 بكسل إلى الشاشات الكبيرة، وتباين ألوان كاف، وتنقل بلوحة المفاتيح، وعناصر نموذج موصوفة بشكل صحيح.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

حسن التصميم المتجاوب وإمكانية الوصول.

التجاوب:

ابدأ من الهاتف ثم وسع، لا العكس
اختبر عند 320 و 375 و 768 و 1280 بكسل
لا تمرير أفقي عند أي عرض
مساحة اللمس لأي زر أو رابط لا تقل عن 44 بكسل
استخدم الخصائص المنطقية في CSS حتى يعمل التخطيط في الاتجاهين

إمكانية الوصول:

تباين النص مع الخلفية بنسبة 4.5 إلى 1 على الأقل للنص العادي
كل صورة لها نص بديل يصف محتواها، والزخرفية نصها البديل فارغ
كل حقل نموذج مرتبط بعنوان label صريح
مؤشر تركيز ظاهر عند التنقل بلوحة المفاتيح
ترتيب العناوين منطقي بلا قفزات
حالة الخطأ في النموذج معلنة لقارئ الشاشة

قواعد:

لا تستخدم اللون وحده لنقل معلومة
لا تلغ مؤشر التركيز الافتراضي دون بديل أوضح منه

اعرض قائمة بما ستغيره قبل أن تنفذ.

الملفات المتوقعة إنشاؤها أو تعديلها

- ملفات الأنماط
- مكونات التنقل والبطاقات والنموذج
- `app/layout` إن احتاج ضبطاً إضافياً

كيف تختبر بنفسك

1. افتح الموقع على هاتف حقيقي لا على محاكي فقط.
2. تنقل في الموقع بمفتاح Tab وحده: هل تصل إلى كل عنصر وترى أين أنت؟
3. كبر حجم خط المتصفح إلى 200% وتأكد أن التخطيط لا ينكسر.
4. شغل فحص إمكانية الوصول في أدوات المطور وأصلح ما يظهر.
5. اطلب من شخص لم ير الموقع أن ينجز مهمة البحث والتواصل، وراقب أين يتعثر.

أخطاء محتملة

- استخدام `margin-left` بدل `margin-inline-start`، فينكسر التخطيط في RTL.
- إلغاء مؤشر التركيز لأنه «غير جميل»، فيصبح الموقع غير قابل للاستخدام بلوحة المفاتيح.
- نص رمادي فاتح على أبيض: يبدو أنيقاً وغير مقروء.

معييار الانتقال

لا تمرير أفقي عند 320 بكسل، والتنقل بلوحة المفاتيح يعمل، والتباين مقبول.

الأداء وأساسيات SEO

الموقع البطيء يترك قبل أن يقرأ. والموقع الذي لا يوصف بشكل صحيح لا يوجد.

ما الذي نبنيه في هذه المرحلة

صفحات أسرع وأخف، وبيانات وصفية صحيحة لكل صفحة، وخريطة موقع، وصور محسنة.

ما ترسله إلى Claude Code

أرسل هذا كما هو بعد تعبئة المتغيرات

حسن الأداء وأساسيات SEO.

الأداء:

حسن كل الصور: أبعاد محددة، وصيغة حديثة، وتحميل كسول لما هو خارج الشاشة
راجع ما يرسل من JavaScript إلى المتصفح، وقلل ما يمكن تقليله
تأكد أن الخطوط لا تسبب وميض نص غير منسق
تجنب انزياح التخطيط: احجز مساحة لكل صورة وعنصر يحمل لاحقاً

SEO الأساسي:

عنوان ووصف فريدان لكل صفحة، بالعربية
بيانات وصفية للمشاركة على الشبكات الاجتماعية
عنوان رابط قانوني لكل صفحة
خريطة موقع sitemap وملف robots
لغة الصفحة معلنة بشكل صحيح
بيانات منظمة للمزود على صفحة التفاصيل

قواعد:

لا تحشو كلمات مفتاحية
لا تعد بشيء في العنوان لا تقدمه الصفحة
اعرض قائمة بما ستغيره قبل التنفيذ.

الملفات المتوقعة إنشاؤها أو تعديلها

- بيانات وصفية في كل صفحة
- `app/robots` و `app/sitemap`
- مكون الصورة في البطاقات وصفحة التفاصيل

كيف تختبر بنفسك

1. شغل فحص الأداء في أدوات المطور على وضع الهاتف مع شبكة بطيئة.
2. افتح كل صفحة وتحقق أن العنوان والوصف مختلفان ومناسبان.
3. شارك رابط صفحة تفاصيل في تطبيق محاكاة وانظر شكل المعاينة.
4. افتح خريطة الموقع وتأكد أنها تحتوي المسارات الفعلية.
5. راقب انزياح التخطيط أثناء تحميل الصفحة.

أخطاء محتملة

- صور بأحجام أصلية ضخمة تصغر بالعرض فقط.
- عنوان ووصف واحد مكرر على كل الصفحات.
- خريطة موقع تحتوي مسارات غير موجودة.

معييار الانتقال

الصفحة الرئيسية تحمل بسرعة مقبولة على شبكة بطيئة، وكل صفحة لها عنوان ووصف فريدان.

الاختبارات والأمان

قبل أن ينشر الموقع لأي أحد، هاتان المرحلتان تحددان إن كان جاهزا فعلا.

الاختبارات

برومبت الاختبارات

أضف اختبارات لهذا المشروع.

الأولوية بهذا الترتيب:

أولا. طبقة البيانات: البحث، والفلتر، والترقيم، والمزود غير الموجود، والنتائج الفارغة
ثانيا. قواعد التحقق: كل قاعدة، وكل حالة حدية، وكل حالة يجب أن ترفض
ثالثا. تحويل البيانات وأي منطق حسابي

لكل اختبار:

اسم يصف السلوك المتوقع لا اسم الدالة
اختبار واحد لسلوك واحد
لا اعتماد على ترتيب التنفيذ ولا على ملفات حقيقية

ثم:

اذكر ما لم تختبره ولماذا
اذكر أي جزء صعب اختباره، وما التغيير في التصميم الذي يسهله
أضف أمر تشغيل الاختبارات إلى CLAUDE.md

استخدم أداة اختبار مستقرة ومتوافقة مع إعداد المشروع.

الأمان

- الأسرار: لا مفتاح في الكود. `.env` مستبعد من Git. المتغيرات التي تصل للمتصفح لا تحتوي أسراراً.
- المدخلات: كل مدخل يتحقق منه على الخادم، ويعقم قبل العرض.
- رسائل الخطأ: مفهومة للمستخدم، وتفاصيلها في السجل لا على الشاشة.
- الاعتماديات: راجع تحذيرات الأمان في مدير الحزم وحدث ما يلزم.
- الحد من الإساءة: حد لعدد الطلبات من المصدر نفسه على نقطة إرسال النموذج.

راجع هذا المشروع أمنيا قبل النشر.

افحص:

أي سر أو مفتاح مكتوب في الكود أو في متغير يصل إلى المتصفح
كل مدخل من المستخدم: هل يتحقق منه على الخادم
كل مكان يعرض فيه نص من المستخدم: هل يعقم
رسائل الخطأ الظاهرة: هل تكشف تفاصيل داخلية
الاعتماديات: هل فيها تحذيرات أمنية معروفة
نقطة إرسال النموذج: هل يمكن إساءة استخدامها بالإرسال المتكرر

لكل ملاحظة: الخطورة، والموضع، وسيناريو الاستغلال، والإصلاح المقترح.

رتب من الأخطر.

إذا كان الكود غير كاف للحكم على نقطة، قل ذلك بدل التخمين.

قاعدة قبل النشر

شغل `git log -p` وابحث عن أي مفتاح أو كلمة مرور مرت في تاريخ المستودع. إذا وجدت واحدا، أبطله وأنشئ غيره — حذفه في commit جديد لا يحذفه من التاريخ.

البناء والنشر والصيانة

الموقع الذي يعمل على جهازك ليس موقعاً. الموقع هو ما يفتحه شخص آخر من هاتفه.

قبل النشر

1. بناء نظيف من مستودع نظيف، لا من جهازك بحالته الحالية.
2. كل الاختبارات تمر، والفحص بلا أخطاء.
3. متغيرات البيئة موثقة في `.env.example` ومضبوطة في بيئة النشر.
4. صفحة 404 وصفحة الخطأ تعملان.
5. خطة تراجع: كيف ترجع للإصدار السابق خلال دقائق.

أوامر التحقق قبل النشر

```
npm run lint
npm test
npm run build
npm start
```

برومبت تجهيز النشر

أرسل هذا قبل النشر

جهز هذا المشروع للنشر.

المطلوب:

أولاً. تحقق أن البناء ينجح من نسخة نظيفة
ثانياً. اذكر كل متغير بيئة يحتاجه المشروع، وأين يضبط في بيئة النشر
ثالثاً. راجع ما يصل إلى المتصفح: هل هناك متغير حساس مكشوف
رابعاً. اكتب README فيه: ما هو المشروع، وكيف يشغل محلياً، وكيف يبني، وكيف ينشر
خامساً. قائمة تحقق قبل كل نشر مستقبلي

لا تنفذ النشر. جهز فقط، وأنا أنشر.

خيارات النشر

الخيار	مق يناسبك
منصة استضافة متكاملة	أسرع طريق: ربط بالمستودع ونشر تلقائي عند كل دفع
خادم افتراضي تديره	عندما تحتاج تحكما كاملا أو قيودا خاصة بالاستضافة
حاوية	عندما تريد بيئة متطابقة بين التطوير والإنتاج

للمشروع الأول

اختر منصة استضافة متكاملة مرتبطة بمستودعك. إدارة خادم بنفسك مهارة مستقلة تستحق التعلم لاحقا، لكنها ستستهلك وقتك الآن في مسائل لا علاقة لها بموقعك.

بعد الإطلاق

- راقب الأخطاء الحقيقية من المستخدمين، لا من جهازك.
- راجع طلبات التواصل الواردة: هي أول مصدر لملاحظات حقيقية.
- حدث الاعتماديات دوريا، وشغل الاختبارات بعد كل تحديث.
- احتفظ بنسخة احتياطية من البيانات، واختبر الاستعادة مرة على الأقل.
- لا تضيف ميزة لأنك تستطيع. أضفها لأن مستخدما احتاجها.

ورقة المراجعة السريعة

المراحل العشر ومعايير الانتقال

المرحلة	معايير الانتقال
صفر - النطاق	REQUIREMENTS.md فيه قائمة خارج النطاق أطول من قائمة داخله
واحد - البنية	ARCHITECTURE.md يحدد الحالات الأربع لكل صفحة
اثنان - ال Stack	قرار مكتوب بسببه، وإصدارات مستقرة متحقق منها
ثلاثة - البيئة	الموقع يفتح، RTL يعمل، المستودع نظيف
أربعة - الصفحات	كل المسارات تعمل بحالاتها الأربع
خمسة - البيانات	تغيير المصدر يتم في ملف واحد
ستة - النماذج	التحقق يعمل على الخادم مع تعطيل JavaScript
سبعة - التجاوب والوصول	لا تمرير أفقي عند 320 بكسل، والتنقل بلوحة المفاتيح يعمل
ثمانية - الأداء و SEO	عنوان ووصف فريدان لكل صفحة، وتحميل مقبول على شبكة بطيئة
تسعة - الاختبارات والأمان	الاختبارات تمر ولا ثغرة حرجة مفتوحة
عشرة - النشر	شخص آخر يفتح الموقع من هاتفه وينجز المهمة

```
git checkout -b phase/NN-name
claude --permission-mode plan
# review the plan, approve, then implement
/diff
npm run lint && npm test
git add -A && git commit
git switch main && git merge phase/NN-name
```

أخطاء شائعة في أول موقع

- البدء بالكود قبل تحديد النطاق.
- بناء الحالة الطبيعية فقط بلا حالات تحميل وخطأ وفراغ.
- قراءة البيانات مباشرة من الصفحات بلا طبقة وسيطة.
- الاعتماد على تحقق المتصفح وحده.
- استخدام `margin-left` بدل الخصائص المنطقية في موقع RTL.
- نشر بلا خطة تراجع.
- إضافة ميزات بعد الإطلاق بلا طلب من مستخدم حقيقي.

ما تضعه في CLAUDE.md لهذا المشروع

CLAUDE.md

دليل الخدمات

أوامر

تشغيل: npm run dev

بناء: npm run build

اختبارات: npm test

فحص: npm run lint

قرارات

الموقع عربي واتجاهه من اليمين إلى اليسار على مستوى المستند.

استخدم الخصائص المنطقية في CSS لا left و right.

الصفحات لا تقرأ مصدر البيانات مباشرة، بل تستدعي lib/data فقط.

كل تحقق من المدخلات يتكرر على الخادم مهما كان تحقق المتصفح.

لا any في الأنواع.

أعراف

أسماء المكونات بصيغة PascalCase، وأسماء الملفات المساعدة بصيغة kebab-case.

كل نص ظاهر للمستخدم بالعربية.

الخلاصة والخطوة التالية

بنيت موقعاً حقيقياً: بمتطلبات مكتوبة، وبنية مقررة، وطبقة بيانات نظيفة، ونموذج آمن، وتصميم يعمل على الهاتف، ونشر بخطة تراجع. الأهم أنك تعرف لماذا كل قرار اتخذ.

ماذا تبني بعده

1. أضف لوحة إدارة بسيطة لإضافة المزودين بدل تعديل ملف البيانات يدوياً.
2. انقل البيانات إلى قاعدة بيانات حقيقية — طبقة `lib/data` تجعل ذلك تغييراً في ملف واحد.
3. أضف مصادقة إن احتاج مشروعك حسابات، وابدأ بحل جاهز موثوق لا بحل من عندك.
4. أعد بناء المشروع نفسه بإطار آخر لتفهم ما الذي كان من الإطار وما كان منك.

بقية السلسلة

- Claude Code من الصفر إلى الاحتراف — لإتقان الأداة نفسها.
- تعلم البرمجة مع Claude Code — إن شعرت أن الأساسيات ناقصة.
- ابن تطبيقك الأول مع Claude Code — الانتقال من الويب إلى الهاتف.

تنبيه عن الإصدارات

إصدارات Node.js و Next.js و React المذكورة تم التحقق منها من مصادرها الرسمية بتاريخ 6 سبتمبر 2026. أطر الويب تتغير بسرعة: تحقق من أحدث إصدار مستقر قبل أن تبدأ، ولا تبني على إصدار موسوم beta أو rc أو canary.

برومبتات عربية

هذا المنتج من إنتاج قناة برومبتات عربية.
لمتابعة الأدلة والبرومبتات الجديدة:

<https://t.me/PromptsArabic>